



Python Programming Fundamentals

Parameters and Return Values

Programming Fundamentals Team

SETU

2026-09-17



Outline

- 1. Positional Parameters**
- 2. Keyword Arguments**
- 3. Default Parameter Values**
- 4. The return Statement**
- 5. Multiple Return Values**
- 6. Conditional Returns**
- 7. Designing Good Functions**



Outline

1. Positional Parameters

2. Keyword Arguments

3. Default Parameter Values

4. The return Statement

5. Multiple Return Values

6. Conditional Returns

7. Designing Good Functions



1. Positional Parameters

Parameters are variables that receive values when the function is called. **Arguments** are the values passed by the caller.

```
def greet(name, time_of_day):      # name, time_of_day are parameters
    print(f"Good {time_of_day}, {name}!")

greet("Alice", "morning")          # positional – order matters
greet("Bob", "evening")
```



1. Positional Parameters

Positional arguments are matched to parameters **in order** — the first argument goes to the first parameter, the second to the second, and so on.

```
def power(base, exponent):  
    return base ** exponent  
  
print(power(2, 8))      # 2^8 = 256  
print(power(8, 2))      # 8^2 = 64 (different order, different result!)
```



Outline

1. Positional Parameters
- 2. Keyword Arguments**
3. Default Parameter Values
4. The return Statement
5. Multiple Return Values
6. Conditional Returns
7. Designing Good Functions



2. Keyword Arguments

Keyword arguments are passed by **name**, so order doesn't matter.

```
def create_user(username, email, age):  
    print(f"User: {username}, Email: {email}, Age: {age}")  
  
# Keyword arguments – any order  
create_user(email="alice@example.com", age=25, username="alice99")
```



2. Keyword Arguments

Keyword arguments are passed by **name**, so order doesn't matter.

```
def create_user(username, email, age):  
    print(f"User: {username}, Email: {email}, Age: {age}")  
  
# Keyword arguments – any order  
create_user(email="alice@example.com", age=25, username="alice99")
```

You can **mix** positional and keyword arguments — positional must come first:

```
create_user("alice99", age=25, email="alice@example.com") # OK  
# create_user(username="alice99", "alice@example.com", 25) # SyntaxError
```




Outline

1. Positional Parameters

2. Keyword Arguments

3. Default Parameter Values

4. The return Statement

5. Multiple Return Values

6. Conditional Returns

7. Designing Good Functions



3. Default Parameter Values

Give a parameter a **default value** so it becomes optional at the call site.

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Alice")           # Hello, Alice!
greet("Bob", "Hi")       # Hi, Bob!
greet("Carol", greeting="Good morning")  # Good morning, Carol!
```



Outline

1. Positional Parameters
2. Keyword Arguments
3. Default Parameter Values
- 4. The return Statement**
5. Multiple Return Values
6. Conditional Returns
7. Designing Good Functions



4. The return Statement

return sends a value back to the caller and **immediately exits** the function.

```
def square(n):  
    return n * n  
  
result = square(7)      # result = 49  
print(square(5) + 1)    # 26  
print(square(square(3)))# 81
```



4. The return Statement

A function can have **multiple return statements** — the first one reached exits the function:

```
def absolute_value(n):  
    if n < 0:  
        return -n          # exits here for negatives  
    return n               # exits here for non-negatives  
  
print(absolute_value(-5))  # 5  
print(absolute_value(7))  # 7
```



Outline

1. Positional Parameters
2. Keyword Arguments
3. Default Parameter Values
4. The return Statement
- 5. Multiple Return Values**
6. Conditional Returns
7. Designing Good Functions



5. Multiple Return Values

Python functions can return **multiple values** as a tuple.

```
def min_max(numbers):  
    """Return the minimum and maximum of a list."""  
    return min(numbers), max(numbers)  
  
lo, hi = min_max([3, 1, 7, 4, 2, 8, 5])  
print(f"Min: {lo}, Max: {hi}")    # Min: 1, Max: 8
```



5. Multiple Return Values

Tuple unpacking on the left side:

```
def divide_with_remainder(a, b):  
    quotient = a // b  
    remainder = a % b  
    return quotient, remainder  
  
q, r = divide_with_remainder(17, 5)  
print(f"17 ÷ 5 = {q} remainder {r}")    # 17 ÷ 5 = 3 remainder 2  
  
# Or get the full tuple:  
result = divide_with_remainder(100, 7)  
print(result)    # (14, 2)
```




Outline

1. Positional Parameters
2. Keyword Arguments
3. Default Parameter Values
4. The return Statement
5. Multiple Return Values
- 6. Conditional Returns**
7. Designing Good Functions



6.1 Ternary Expression

A **ternary expression** returns one of two values on a single line:

```
return <value_if_true> if <condition> else <value_if_false>
```



6.1 Ternary Expression

```
def is_even(n):  
    return "even" if n % 2 == 0 else "odd"
```

```
def absolute_value(n):  
    return n if n >= 0 else -n
```

```
print(is_even(4))          # even  
print(is_even(7))          # odd  
print(absolute_value(-5))  # 5
```



6.1 Ternary Expression

```
def is_even(n):  
    return "even" if n % 2 == 0 else "odd"
```

```
def absolute_value(n):  
    return n if n >= 0 else -n
```

```
print(is_even(4))          # even  
print(is_even(7))          # odd  
print(absolute_value(-5))  # 5
```

Use the ternary when there are exactly **two** outcomes and both fit clearly on one line. For three or more branches, use chained returns instead (see next slide).



6.2 Chained Early Returns

When a function has **multiple branches**, return from each one immediately — no `elif` needed. The first return reached exits the function:

```
def calculate_grade(score):  
    """Return a letter grade for a numeric score."""  
    if score >= 90: return "A"  
    if score >= 80: return "B"  
    if score >= 70: return "C"  
    if score >= 60: return "D"  
    return "F"           # fallback – only reached if all above are False  
  
print(calculate_grade(95))    # A  
print(calculate_grade(73))    # C  
print(calculate_grade(42))    # F
```



6.2 Chained Early Returns

Why not `elif`? Once a return fires, the function is already finished — Python never reaches the next `if`. This means `elif` is unnecessary, and removing it makes the logic easier to scan top to bottom.



6.2 Chained Early Returns

Why not `elif`? Once a return fires, the function is already finished — Python never reaches the next `if`. This means `elif` is unnecessary, and removing it makes the logic easier to scan top to bottom.

The final bare return `"F"` is the **catch-all** — it only runs if every condition above was `False`.



Outline

1. Positional Parameters
2. Keyword Arguments
3. Default Parameter Values
4. The return Statement
5. Multiple Return Values
6. Conditional Returns
- 7. Designing Good Functions**



7. Designing Good Functions

Good functions:

- Do **one thing** well (single responsibility)
- Have a clear, descriptive name
- Have a docstring
- Accept inputs as parameters
- Return output via `return` (don't print inside unless displaying is the function's job)



7. Designing Good Functions

BAD – function does too many things and mixes I/O with logic

```
def process():  
    name = input("Name: ")  
    score = float(input("Score: "))  
    grade = "Pass" if score >= 60 else "Fail"  
    print(f"{name}: {grade}")
```

GOOD – separate concerns

```
def get_input(prompt):  
    return input(prompt)  
  
def calculate_grade(score):  
    return "Pass" if score >= 60 else "Fail"  
  
def display_result(name, grade):
```



7. Designing Good Functions

```
print(f"{name}: {grade}")

name = get_input("Name: ")
score = float(get_input("Score: "))
grade = calculate_grade(score)
display_result(name, grade)
```



Thanks for Watching - Any questions?

